

Introduction to Algorithms

Levon Prazyan

Course Overview

- **Introduction**
 - 1 lecture
- **Analyzing Algorithms**
 - 2 lectures
- **Recurrences**
 - 2 lectures
- **Probabilistic Analysis and Randomized Algorithms**
 - 2 lecture
- **Sorting and Searching**
 - 4 lectures
- **Heaps**
 - 1 lecture
- **Arrays, Lists**
 - 1 lectures
- **Stack, Queue, Double Ended Queue**
 - 2 lectures

Introduction to Algorithms: Goals and Objectives

- Formalization of the concept of "algorithm"
- Analyzing time and space complexity of algorithms;
- Understand the basic concepts and principles of various data structures.
- Designing efficient algorithms for sorting, searching, and selection problems.
- Analyze the time and space complexity of algorithms involving different data structures

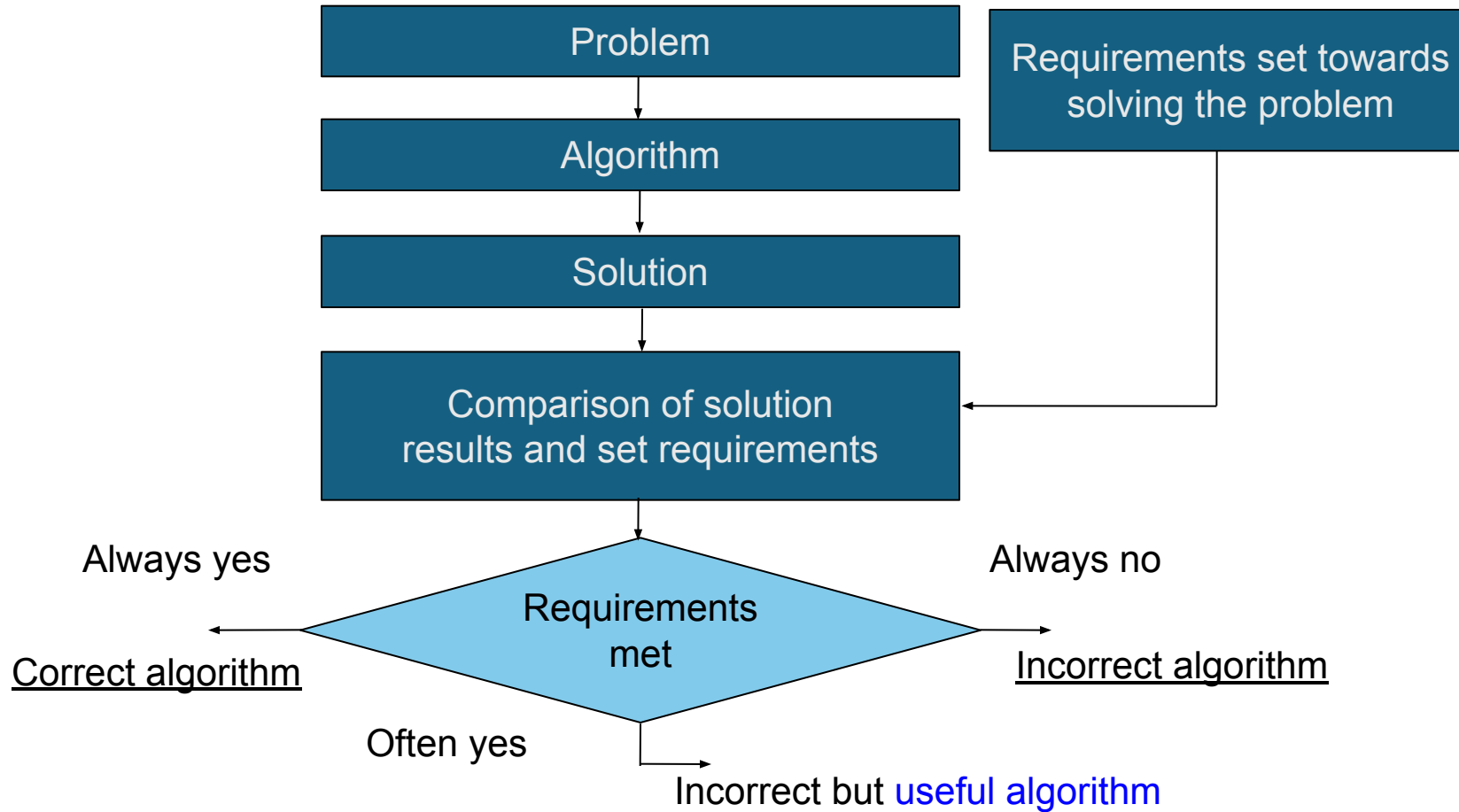
Introduction to Algorithms : Definition

- **Algorithm** - a finite set of clear, unambiguous instructions, due to which a certain output result can be obtained using input data of a definite type.
- The theory of algorithms is a science that studies the general properties and patterns of algorithms and various formal models for their representation.
- Several examples:
 - An algorithm to add numbers in a column
 - Various algorithms to construct with compass and ruler
 - An algorithm to solve quadratic equations

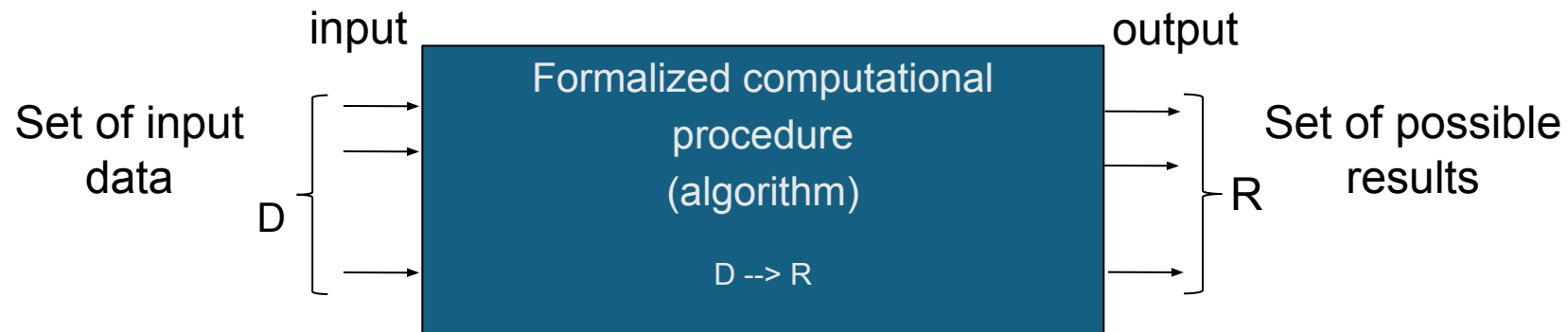
Formalization of Algorithm Concept

- An **algorithm** is a finite instruction given at some language that specifies a finite sequence of elementary operations' feasible solutions to problems, common to the class of possible input data.
- Let D be the region (set) of input data of the problem, and R - the set of possible outcomes, then it can be stated that the algorithm reflects the $D \rightarrow R$. Since such reflection can be incomplete, the following concepts are inserted:
 - An algorithm is called **partial** if the result is obtained only for some $d \in D$ and **complete** if the algorithm gets the correct result for all $d \in D$.

Correct and Incorrect Algorithms



Formalization of the Concept of Algorithm (2)



For example, sorting problem

Input: sequence of n numbers $a_1, a_2, \dots, a_n$

Output: such move of $(a_1, a_2, \dots, a_n)$ into $(a_{1k}, a_{2k}, \dots, a_{nk})$ which will satisfy the following condition:

$a_{1k} \leq a_{2k} \leq \dots \leq a_{nk}$
(10, 4, 59, 2, 4, 31,) --> (2, 4, 4, 10, 31, 59)

In general, there are different algorithms of solving the same problem.

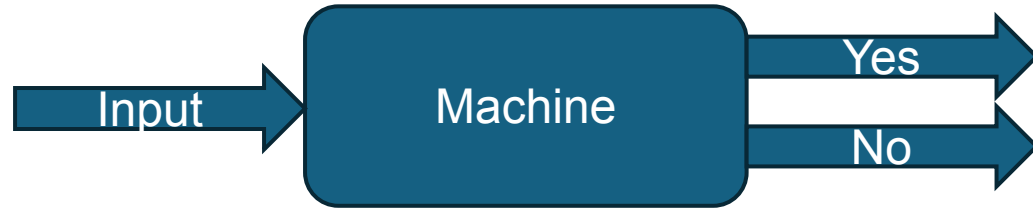
Key Requirements to an Algorithm

- **Correctness**
 - The algorithm must produce the correct output for all possible valid inputs.
- **Efficiency**
 - Time Complexity
 - Space Complexity
- **Scalability**
 - The algorithm should perform well even as the size of the input data grows significantly.
- **Robustness**
 - The algorithm should handle unexpected or invalid inputs gracefully
- **Determinism**
 - Given the same input, the algorithm should always produce the same output and follow the same path of execution.

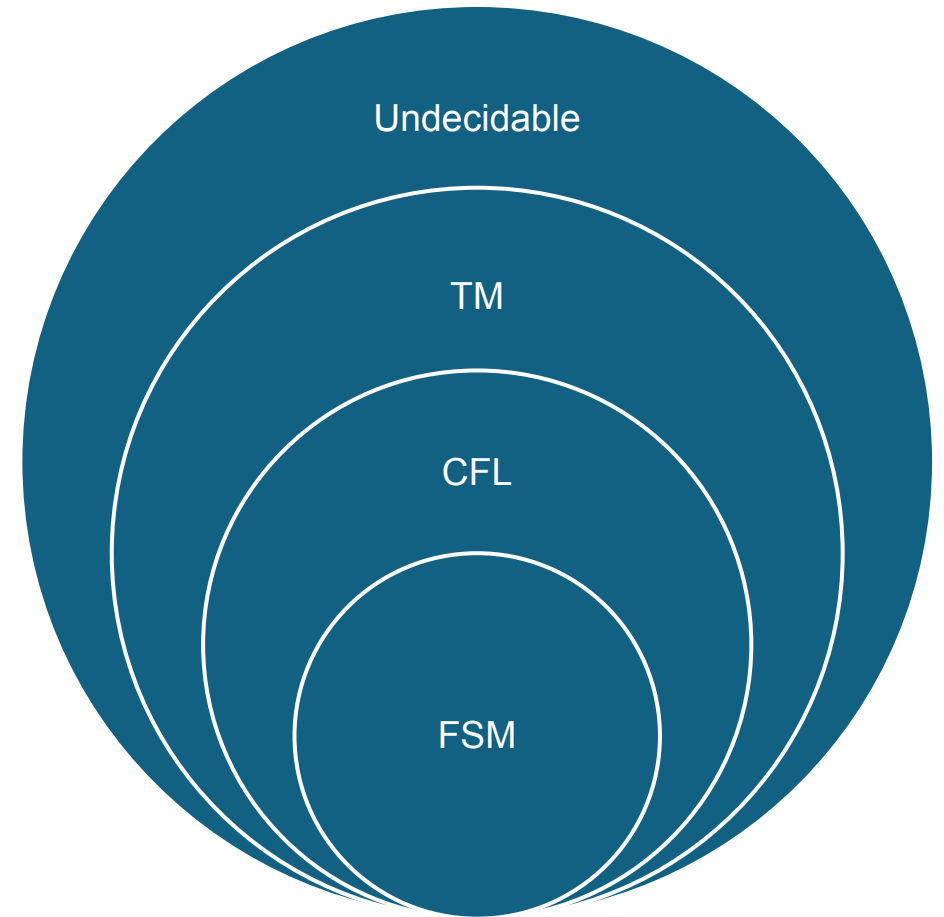
Key Requirements to an Algorithm

- Maintainability
 - The algorithm should be easy to understand, modify, and extend by other developers.
 - Clear documentation.
 - Well-structured code.
- Portability
 - The algorithm should be able to run on different platforms.
- Usability
 - The algorithm should be easy to implement and integrate into larger systems or applications
- Security
 - The algorithm should be secure against potential threats or misuse, particularly if it processes sensitive data.
- Flexibility
 - The algorithm should be adaptable to different use cases or input variations without significant changes.

Levels of Computational Models



- FSM – Finite State Machine
- CFL – Context Free Language
- TM – Turing Machine





Thank You,
Q&A